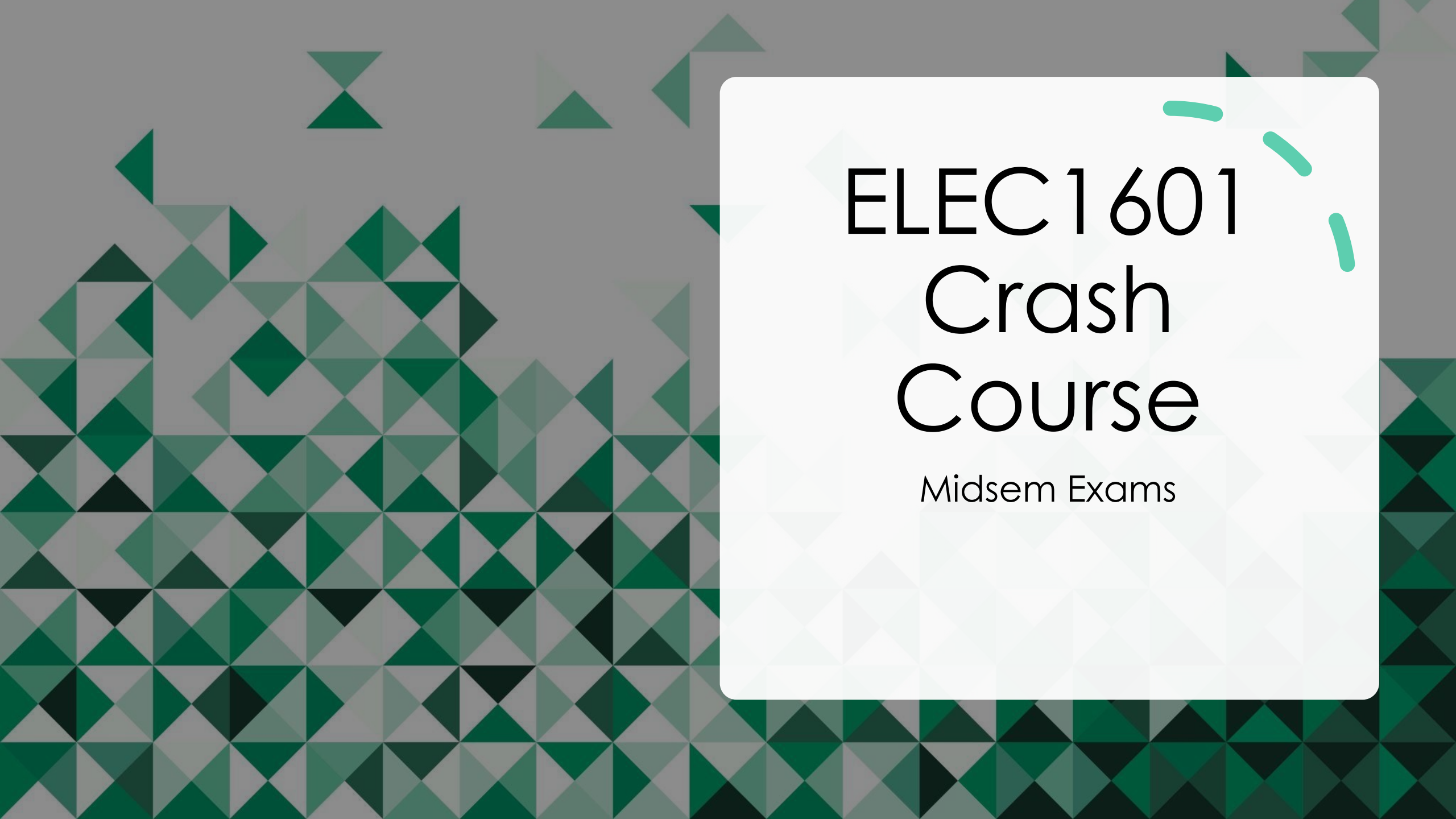




ELEC1601 Crash Course

Midsem Exams

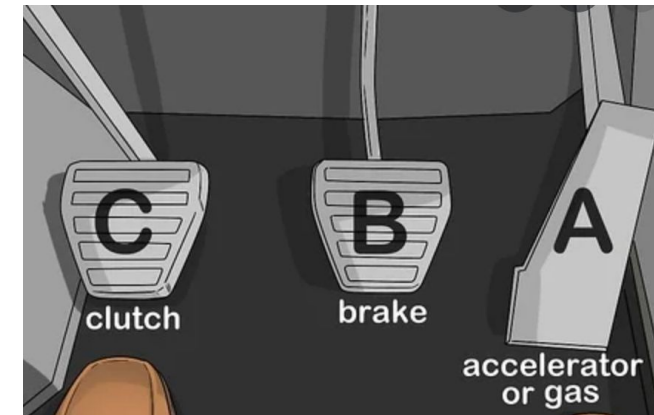
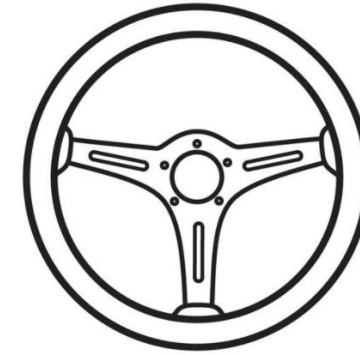
Levels of Abstraction

You shouldn't need to think about the levels below you

Level 6: User	
Level 5: High Level Programming	
Level 4: System	
Level 3: Assembly Language	
Level 2: Microprocessor	
Level 1: Control circuit	
Level 0: Digital Logic	

Transistors, Microcode, Assembly Code, C++, Executable programs, Apps,
Instruction Set Architecture, Driver, Java, Hardwired control, Gates,
Operating System, Circuit, Library code, Silicon Chip

You don't drive consciously thinking about the fuel intake



Two States

1	0
High	Low
On	Off
Two present	No two present

Standard Binary Range

$[0, (2^n - 1)]$

$n = \text{number of elements to encode}$

$$\text{bits} = \log_2(n)$$

$$n = 2^{\text{bits}}$$

We need to round up in both cases

Example:

100 states to encode

$$\text{bits} = \log_2(100) = 6.64$$

We need at least 6.64 bits.

$\therefore$ 7 bits

2's Comp:

First bit is negative weight

1. Flip all the bits
2. Add 1

1's Comp:

Inverted binary

1. Flip all the bits

Both are used to encode negative numbers

Encoding in a Base

Divide by 2 method for binary:

1. Write number
2. Short divide by two
3. Write remainder down
4. Repeat 2 & 3

Example:
Convert 39 to binary

19	1
9	1
4	1
2	0
1	0
0	1

39 = 100111

	Binary	Octal	Hexadecimal	Decimal
Conversion	1 bit	3 bits	4 bits	Calculator
Prefix	0b	0o	0x	0d
Examples	101001	51	29	41
	1011	13	B	11
	10110101001	2651	5A9	1449
	11010110	326	D6	214

Sign Magnitude

Sign **Magnitude**

 Value

Range

$[-(2^{n-1} - 1), (2^{n-1} - 1)]$

Sign Magnitude

40 = 0 101000 = 0101000

-12 = 1 1100 = 11100

-39 = 1 100111 = 1100111

Fixed Point

- Integer bits determine range (choose to cover range)
- Fractional bits determine accuracy

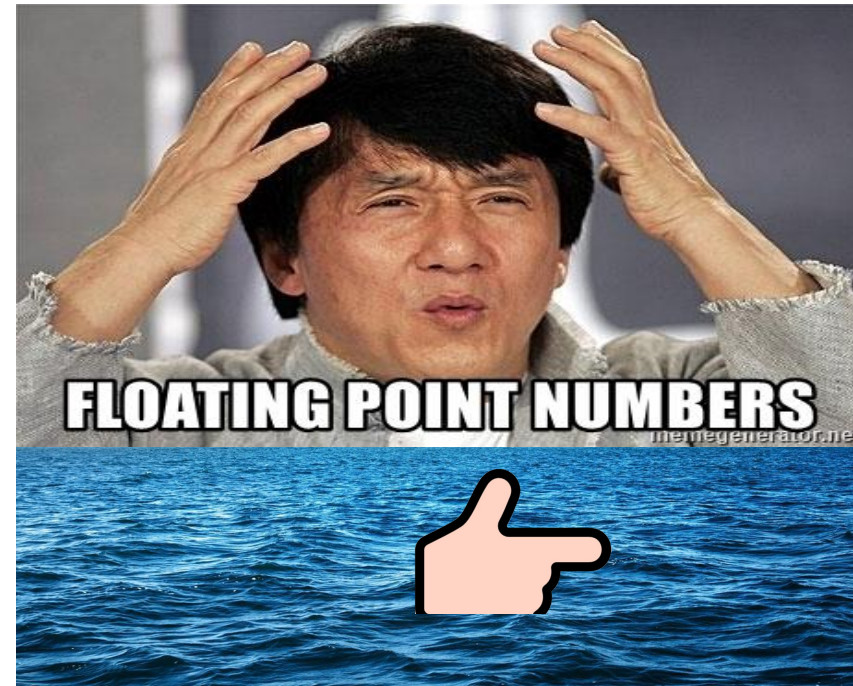
$b_0b_1b_2b_3.b_4b_5b_6b_7 \rightarrow 4 \text{ bit int}, 4 \text{ bit frac.}$

You cannot represent everything perfectly, such as $1/3$.

Floating Point

Sign	Mantissa	Exponent

- *Exponent usually in 2's complement*
- *Choose exponent to cover range*
- Exactly the same as scientific notation
- Binary scientific notation



Bias

Addition bias

- Shift a value by a certain constant K each time (good for floating point particularly)

Multiplication bias

- In terms of powers of two usually (bit shifting)

Memory Operations

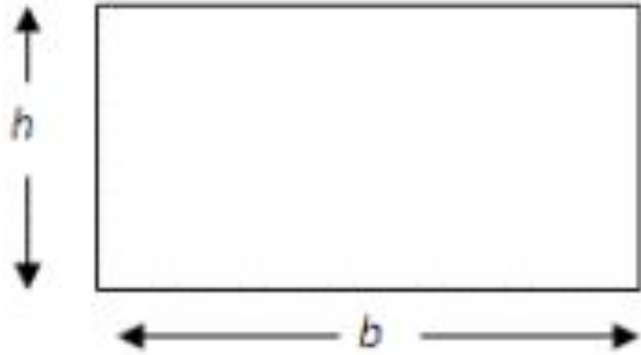
Read

- `read(@)`
- don't know what is stored at addresses when we've just turned on computer (undefined)
- can read in chunks of bytes if specified

Write

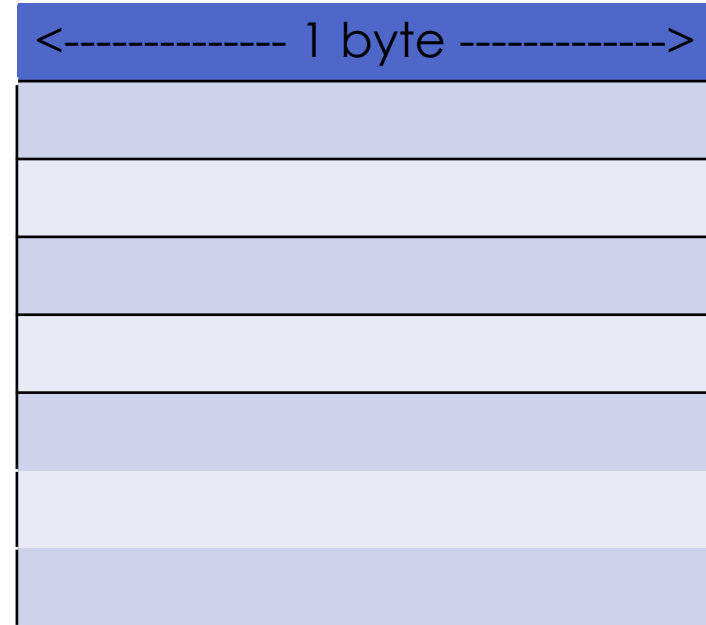
- `write(@, val)`
- can write in chunks of bytes if specified

Memory Size



*bytes = cells * bytes per cell*
*[b] = [c] * [b/c] → units*

0x100
0x101
0x102
0x103
.
.
.



$KB = 2^{10}$
 $MB = 2^{20}$
 $GB = 2^{30}$
 $TB = 2^{40}$

Memory Address Bits

Example:

How many bits will you need to encode your addresses if you have 8KB of storage with 2 byte cells.

$$8 * 2^{10} = \text{cells} * 2$$

$$2^3 * 2^{10} = \text{cells} * 2$$

$$2^{13} = \text{cells} * 2$$

$$\text{cells} = \frac{2^{13}}{2} = 2^{12}$$

$\therefore$ 12 bits

Encoding Different Things

<https://canvas.sydney.edu.au/courses/35882/pages/3-dot-4-data-storage>

Boolean	1 bit
Character	2 bytes
Integer	4 bytes
Long	8 bytes
Float	4 bytes
Double	8 bytes

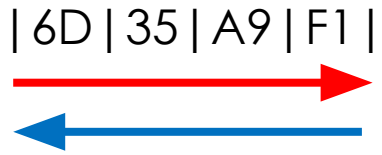
You will likely be asked about a structure with a few different types of things in it.

Little/Big Endian

Example:

Store 0x6D35A9F1

1. Break up into correct size chunks
2. Put into cells



Big Endian

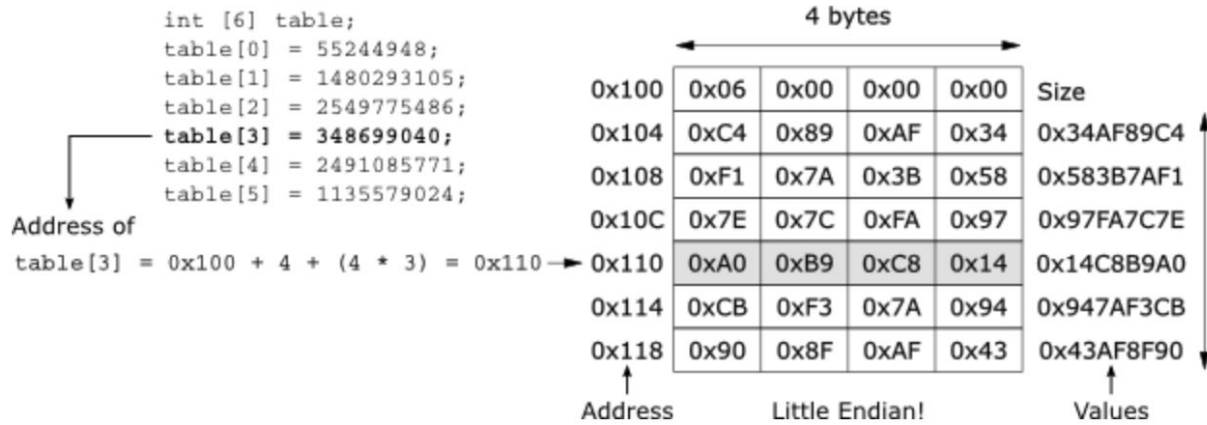
	<----- 1 byte ----->
0x100	0x6D
0x101	0x35
0x102	0xA9
0x103	0xF1
.	
.	
.	

Little Endian

	<----- 1 byte ----->
0x100	0xF1
0x101	0xA9
0x102	0x35
0x103	0x6D
.	
.	
.	

Storing in an Array

https://canvas.sydney.edu.au/courses/35882/pages/3-dot-5-storing-an-array?module_item_id=1278315



A Java array of six integers stored in memory

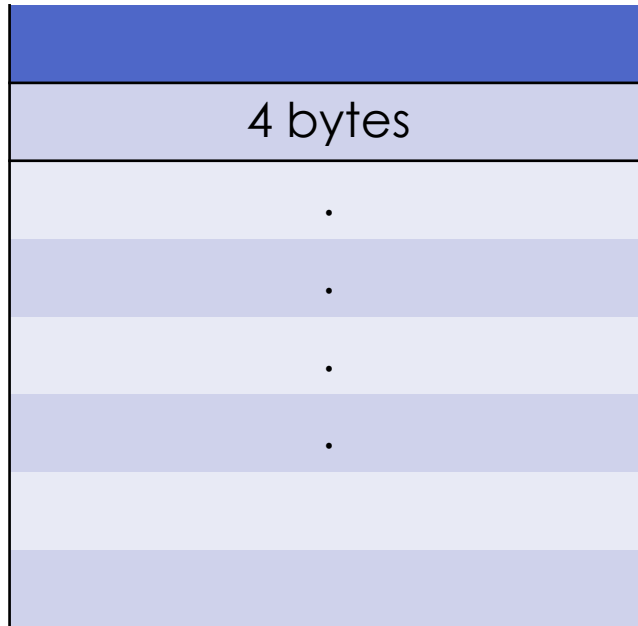
IndexOutOfBounds Error

1. Obtain the integer s stored in position d
2. Check that $0 \leq i$. If not satisfied, raise exception.
3. Check that $i < s$. If not satisfied, raise exception.
4. Calculate the address of the element as $d + 4 + (t * i)$.

This address layout style is not course content, don't freak out if you haven't seen it.

Storing in an Array

Java Array



Size of
the array

d = address (@)
 t = element byte size
 i = index

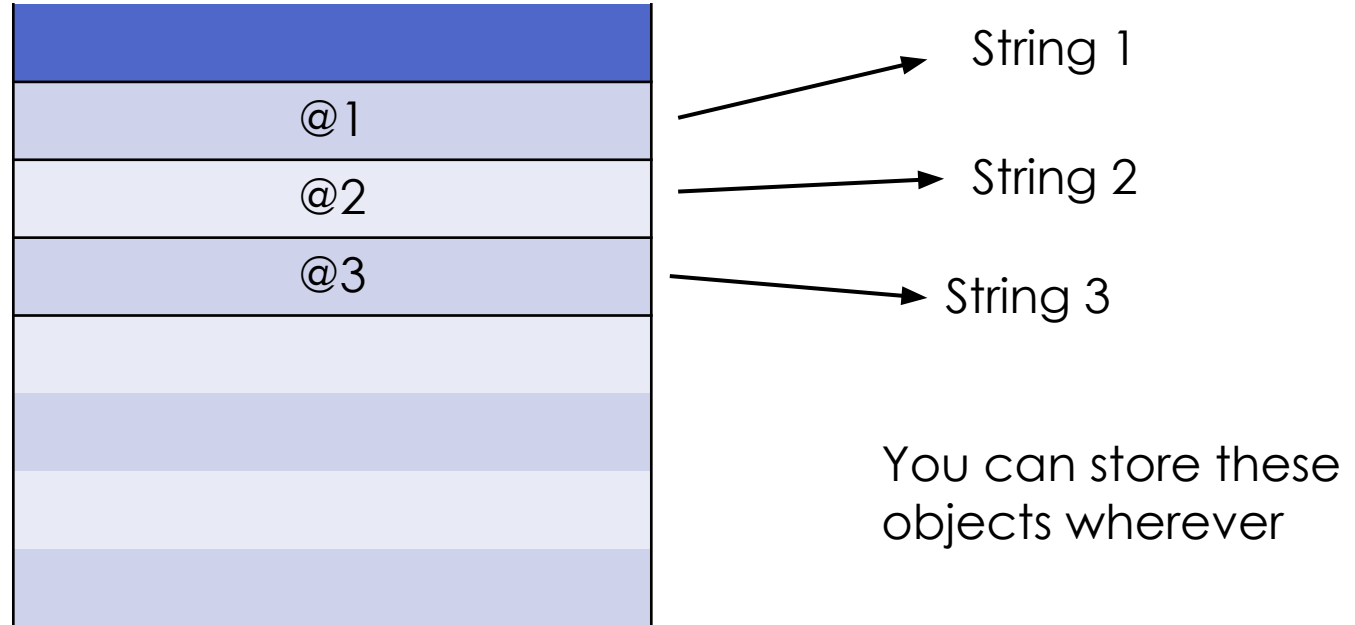
$$\text{Address} = d + 4 + (t * i)$$

Note: the +4 is due to the
initial four bytes in a java array

If not stated, assume each
address is 1 byte memory

Indirection

Indirection: storing a cell address in another cell



Boolean Logic

x -> 0 or 1

y ...

z ...

These can also be thought of as true or false, high or low.

We can do algebra with these kinds of variables.

a	b	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

a	NOT
0	1
1	0

$$a \text{ AND } b = ab = a * b = a \& b$$

$$a \text{ OR } b = a + b = a | b = a || b$$

$$\text{NOT } a = a' = \bar{a}$$

$$a \text{ XOR } b = a \wedge b = a \oplus b$$

Decimal System: $1+1=2$
Binary System: $1+1=10$
Boolean Algebra: $1+1=1$

Non-Programmers:



They have no idea

Simplifying Boolean Expressions

Name	AND Form	OR Form
Identity Law	$1A = A$	$0 + A = A$
Null Law	$0A = 0$	$1 + A = 1$
Idempotent Law	$AA = A$	$A + A = A$
Inverse Law	$A\bar{A} = 0$	$A + \bar{A} = 1$
Commutative Law	$AB = BA$	$A + B = B + A$
Associative Law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive Law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption Law	$A(A + B) = A$	$A + AB = A$
De Morgan's Law	$\overline{AB} = \bar{A} + \bar{B}$	$\overline{(A+B)} = \bar{A}\bar{B}$

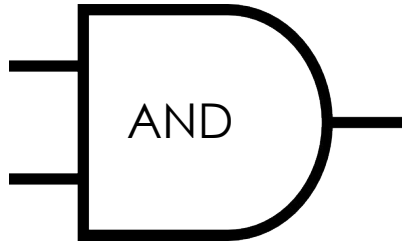
Standard Problem

$$\begin{aligned} & xy + (x' + y)' \\ &= xy + xy' \\ &= x(y + y') \\ &= x(1) \\ &= x \end{aligned}$$

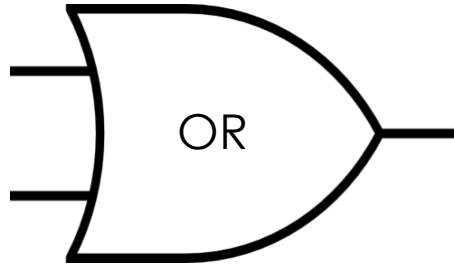
A Little Harder

$$\begin{aligned} & (x \wedge y) + (xy)' + x'y \\ &= x'y + xy' + (x' + y') + x'y \\ &= x'(y + 1) + y'(x + 1) \\ &= x' + y' \text{ full simplification} \\ &= (xy)' \text{ single gate} \end{aligned}$$

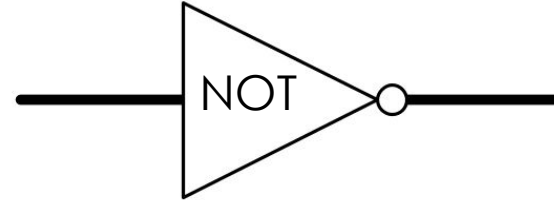
Logic Gates



Are all the
inputs 1

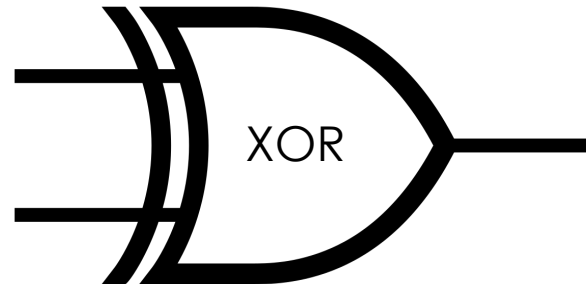


Is there at
least a 1



Is the
input 0

- We can chain multiple gates together to create a logic circuit
- We can chain many of the same gate together to create a 'multi' input equivalent



Do the inputs
add to an
odd number

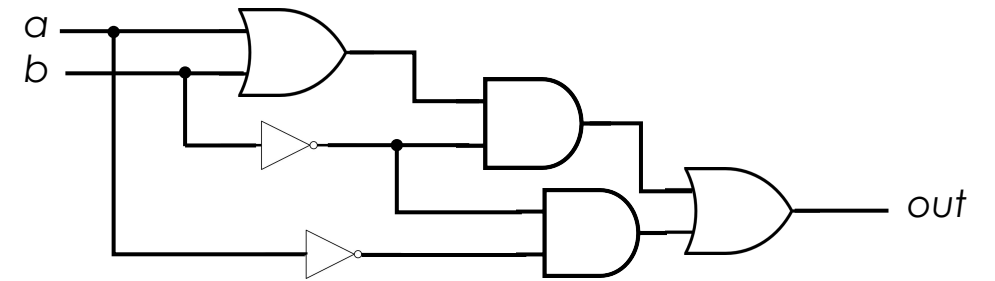
Logic Circuits

Example:

Design a circuit with
 $out = a'b + abc + ac'$

Example:

What is the truth
table for this circuit



Canonical Forms

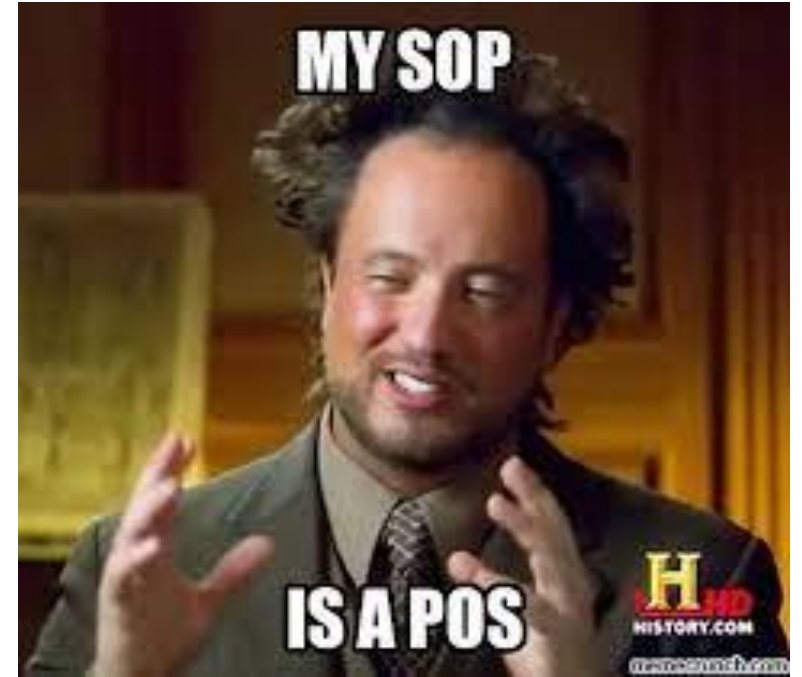
SOP	POS
Sum of Products	Product of sums
For less 1's (more 0's)	For less 0's (more 1's)
Faster to derive	Slower to derive
Much loved	Usually hated

Example from
Tutorial 3 may help

x	y	out
0	0	1
0	1	0
1	0	0
1	1	1

SOP
 $\text{out} = x'y' + xy$

POS
 $\text{out} = (x + y')(x' + y)$



Canonical Forms Example

a	b	c	out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

SOP:

$$1 = X$$

$$0 = \bar{X}$$

$$\text{out} = a'bc' + a'bc + ab'c + abc$$

simplify out of canonical form

$$\text{out} = a'b + ac$$

POS:

$$0 = X$$

$$1 = \bar{X}$$

$$\text{out} = (a + b + c)(a + b + c') \\ (a' + b + c)(a' + b' + c)$$

simplify (much more tedious)

$$\text{out} = a'b + ac$$

Canonical Forms Design Flow

Circuit → Truth Table → Algebra (SOP, POS forms)

Algebra → Truth Table → SOP, POS → Circuit

Canonical Forms Circuit

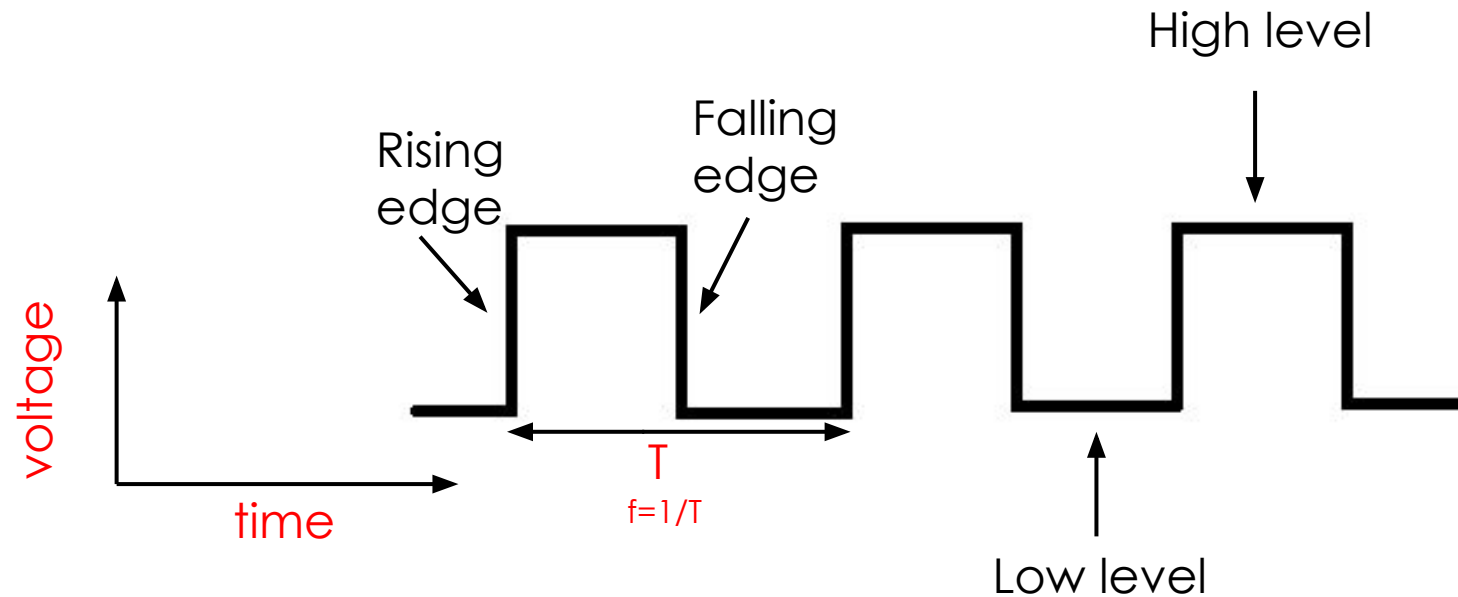
Example:

Design a canonical circuit with this output

$$\text{out} = (a + b + c)(a + b + c')(a' + b + c)(a' + b' + c)$$

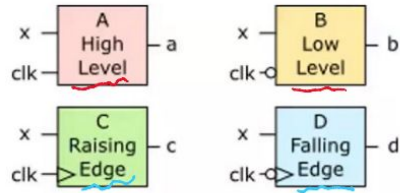
The Clock

Things that happen in circuits are based on a clock so we can organise when we want them to take place and, thus, carry out correct computations.

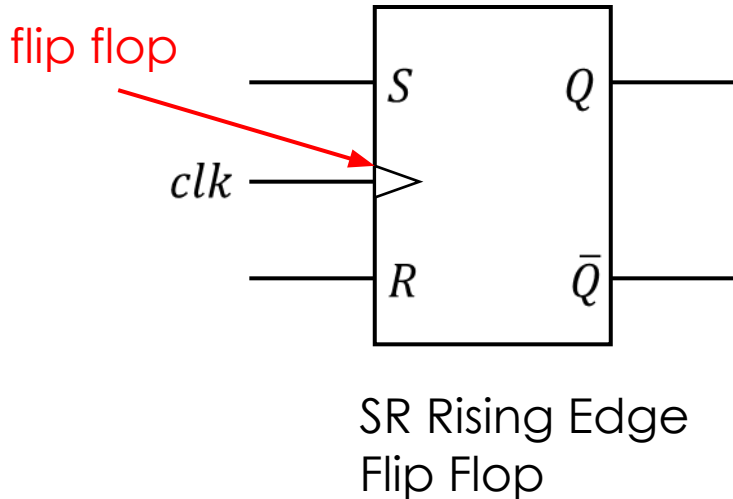


Latches and Flip Flops

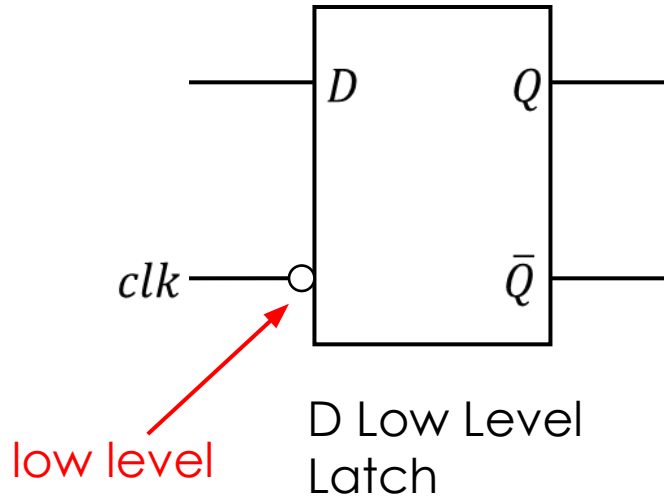
- Used for storing values
- Bubble on clock signal means low level/falling edge
- Triangle means flip flop, no triangle means latch



→ Latches
→ Flip Flop

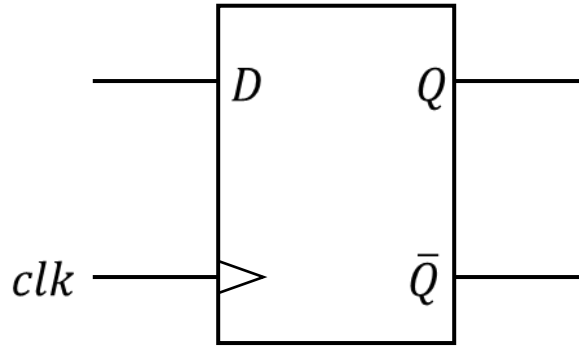


SR Rising Edge
Flip Flop



D Low Level
Latch

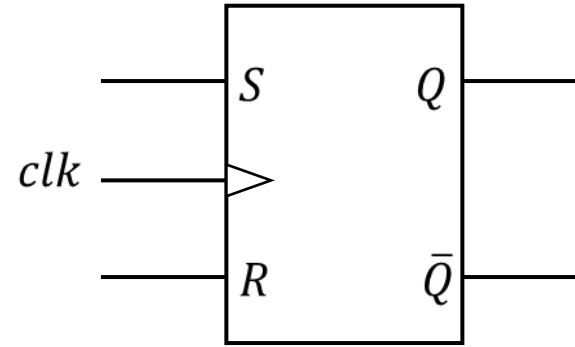
SR, JK, D Flip Flops & Latches



D Flip Flop

D	Q_{t+1}
0	0
1	1

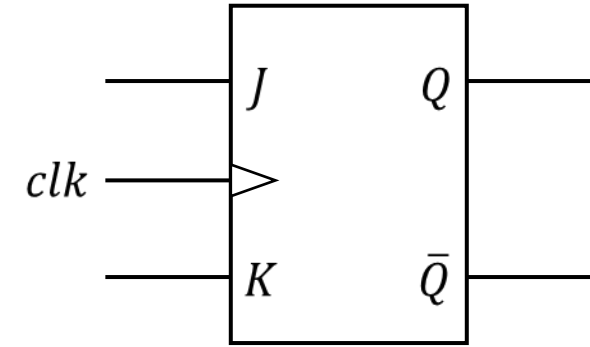
next time
interval



SR Flip Flop

S	R	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	undef.

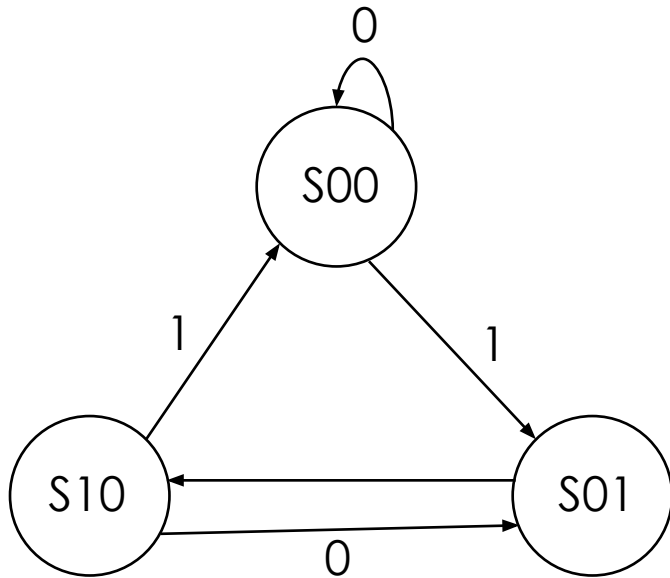
current
time



JK Flip Flop

J	K	Q_{t+1}
0	0	Q_t
0	1	0
1	0	1
1	1	$\bar{Q}_t$

Finite State Machine (FSM)



s_0	s_1	i	$s_0^+ \quad s_1^+$	
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Moore Machine:

- Output solely from state

Mealy Machine:

- Output from state transition

Resources

General:

How Do Computers Work – Sebastian Lague

https://www.youtube.com/watch?v=QZwneRb-zqA&list=PLFt_AvWsXI0dPhqVsKt1Ni_46ARyiCGSq

Computer Systems – Abelardo Pardo

<https://www.youtube.com/watch?v=Z4Zz7n-Lj0g&list=PLpqp4pX4fQmnLXXL7sl4dtLocDOgJUWg0>

Base Conversion

<https://www.rapidtables.com/convert/number/base-converter.html>

2's Complement Calculator

<https://www.exploringbinary.com/twos-complement-converter/>

Boolean Logic Calculator

<https://www.dcode.fr/boolean-expressions-calculator>

Jono's Email (don't freak out if he doesn't answer in time, do some study)

jonathon.priestley@sydney.edu.au

Easier Binary Questions

Cover up answers if you want to test yourself

Convert to binary

Decimal	Binary
10	1010
204	11001100
73	1001001
27	11011

How many bits for this range

Range	Bits
[0,150]	8
[20,80]	6 (bias +20)
[-4,10]	4 (bias -4)
[0,7]	3

Convert to 8 bit 2's comp.

Decimal	2's Comp
-33	11011111
54	00110110
-13	11110011
-119	10001001

What is this number in sign magnitude

Decimal	Sign Mag.
-12	11100
42	0101010
-2	110
-109	11101101

Floating and Fixed Point Questions

Cover up answers if you want to test yourself

8 bit allocation for
Floating Point ranges

Range	S	M	E
[-127,127]	1	2	5
[-63,63]	1	3	4
[-4,11]	1	3	4
[-256,256]	1	2	5

Max precision on
Floating Point

S	M	E	Precision
1	3	4	
1	5	2	
1	1	6	
1	4	3	

Max error on Floating
Point

S	M	E	Error
1	3	4	
1	5	2	
1	1	6	
1	4	3	

8 bit allocation for
Fixed Point ranges

Range	Int	Frac
[0,256]	8	0
[0,63]	6	2
[10,20]	4	4
[-13,60]	7	1

Range for given
Fixed Point styles

Int	Frac	Range
5	1	[0,31.5]
4	2	[0,15.75]
3	3	[0,7.875]
1	1	[0,1.5]

Max error on Fixed
Point

Int	Frac	Error
5	1	
4	2	
3	3	
1	1	

Memory Questions

Cover up answers if you want to test yourself

1. A java structure contains 2 integers, a character, and 3 floats. If the structure address is @0x100, what is the address of the last float?

Answer:

0x100	4 byte size
0x104	int1
0x108	int2
0x10C	char
0x10E	float1
0x112	float2

2. The number 0x32??B6?? is the same in little endian and big endian. What is the number?

Answer:

0x??	←→	0x32
0xB6	←→	0x??
0x??	←→	0xB6
0x32	←→	0x??

3. In a particular architecture, the operations read and write work in 4 byte chunks of big endian. What is the output of the last line below:

```
write(0x100, 0x1B842CD3)
write(0x101, 0x2B852CD3)
write(0x102, 0x3B862CD3)
read(0x100)
```

Answer:

0x100	0x1B
0x101	0x2B
0x102	0x3B
0x103	0x86
0x104	0x2C
0x105	0xD3

read(0x100) -> 0x1B2B3B86

Boolean Logic Questions

Cover up answers if you want to test yourself

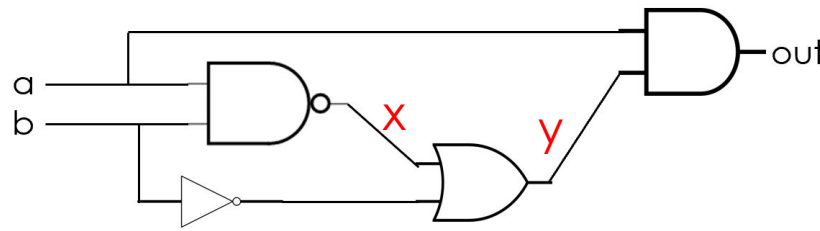
1. Simplify $out = a'bc + abc + abc' + ab$

$$out = bc(a+a') + ab(c'+1)$$

$$out = bc + ab \text{ valid answer}$$

$$out = b(c + a) \text{ also valid answer}$$

2. What is the output expression for this circuit?



$$x = (ab)' = a' + b' \text{ (de morgan's law)}$$

$$y = b' + x = b' + a' + b' = a' + b' \text{ (idempotent law)}$$

$$out = a*y = a(a' + b') \text{ (substitution)}$$

$$out = a*a' + ab' \text{ (distributive law)}$$

$$out = 0 + ab' \text{ (null law)}$$

$$out = ab' \text{ (identity law)}$$

3. Convert to a truth table:
 $out = ac + ab'c' + a'bc'$

a	b	c	out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Canonical Forms Questions

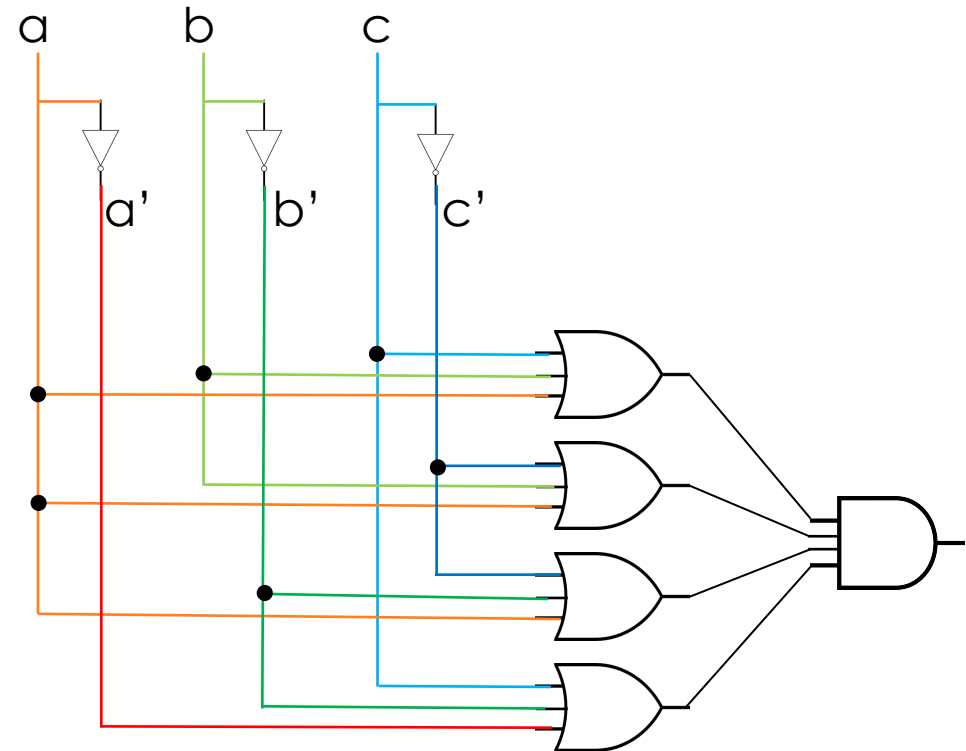
Cover up answers if you want to test yourself

1. What is the POS form of this truth table?

a	b	c	out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

$$\text{out} = (a + b + c)(a + b + c') \\ (a + b' + c')(a' + b' + c)$$

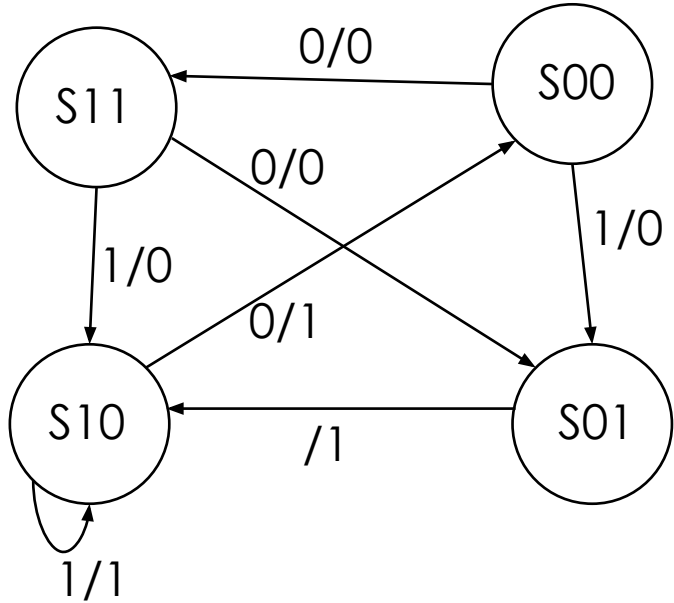
2. Turn the answer from Q1 into a canonical form circuit



Colour coded so you can see what's what. Change back to black if you'd like.

FSM Questions

Cover up answers if you want to test yourself



1. Fill in the truth table for this FSM

s_0	s_1	i	s_0^+	s_1^+	o
0	0	0	1	1	0
0	0	1	0	1	0
0	1	0	1	0	1
0	1	1	1	0	1
1	0	0	0	0	1
1	0	1	1	0	1
1	1	0	0	1	0
1	1	1	1	0	0

2. Is this a Moore or a Mealy Machine?

This is a Moore Machine because every state has the same output for both input options, so the output is based solely on the state.